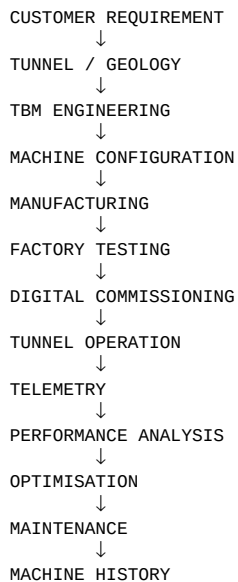


TBM FOUNDRY

Tunnel Boring Machine Engineering, Simulation,
Manufacturing & Digital Twin Platform

Development Model

A high-technical-grade software architecture for a tunnel boring machine manufacturer, combining hard-science Python simulation, Kotlin digital-twin visualisation, Rust real-time infrastructure, R engineering analytics, and PostgreSQL/PostGIS persistence.

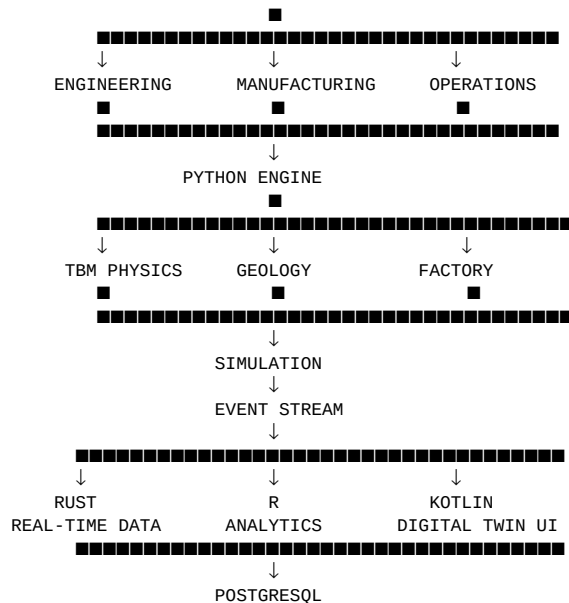


1. Executive Architecture

TBM Foundry is conceived as an industrial digital engineering platform rather than a conventional tunnelling game or isolated simulator. It represents the complete lifecycle of a tunnel boring machine, from customer requirements and engineering configuration through factory manufacture, commissioning, field operation, telemetry, maintenance and subsequent machine development.

- **Python** — hard-science physics, simulation, engineering models and scenario generation.
- **Kotlin** — visualisation, animation, operator and engineering dashboards, and digital-twin presentation.
- **Rust** — real-time telemetry, event infrastructure, high-throughput processing and selected numerical hot paths.
- **R** — statistical engineering, regression, forecasting and performance analysis.
- **PostgreSQL/PostGIS** — persistent system of record and spatial/geological data.

TBM FOUNDRY



2. Development Model

The architecture is deliberately polyglot, with strict separation of concerns.

PYTHON

= PHYSICS + SIMULATION + ENGINEERING MODELS + SCENARIOS

KOTLIN

= VISUALISATION + ANIMATION + OPERATOR / ENGINEERING DASHBOARD

RUST

= REAL-TIME DATA + HIGH-PERFORMANCE SERVICES + TELEMETRY

R

= STATISTICS + ENGINEERING CALCULATORS + PERFORMANCE ANALYSIS

POSTGRESQL

= PERSISTENT SYSTEM OF RECORD

POSTGIS

= SPATIAL / GEOLOGICAL DATA

Python must not become the frontend; Kotlin must not contain the core physics; R must not control real-time machine state; and Rust must not become a monolithic business-logic layer.

3. Python Scientific Engine

The Python layer is the authoritative computational environment for the virtual TBM. It should model coupled physical behaviour rather than a collection of independent calculators.

- TBM mechanics and cutterhead behaviour
- Thrust, torque, power and penetration
- Geology and heterogeneous tunnel faces
- Excavation and muck transport
- Hydraulics and cooling
- Lubrication and thermal behaviour
- Cutter wear and degradation
- Segment handling and installation
- Electrical systems
- Maintenance and failure scenarios
- Manufacturing and factory processes
- Machine productivity and lifecycle economics

Python project structure

```
tbm_foundry/
├── python_engine/
│   ├── pyproject.toml
│   └── tbm_engine/
│       ├── core/
│       ├── tbm/
│       ├── cutters/
│       ├── geology/
│       ├── excavation/
│       ├── hydraulics/
│       ├── muck/
│       ├── segments/
│       ├── electrical/
│       ├── thermal/
│       ├── factory/
│       ├── manufacturing/
│       ├── maintenance/
│       ├── economics/
│       ├── scenarios/
│       ├── optimisation/
│       ├── telemetry/
│       └── api/
└── tests/
    ├── unit/
    ├── integration/
    ├── physics/
    ├── scenarios/
    └── performance/
```

Core simulation relationship

```
GEOLOGY
↓
CUTTING RESISTANCE
↓
TORQUE / THRUST
↓
CUTTERHEAD POWER
↓
EXCAVATION RATE
↓
MATERIAL FLOW
↓
MACHINE LOAD
↓
THERMAL STATE
↓
WEAR
↓
NEXT STATE
```

Mass and energy balances should be explicit wherever physically applicable. Numerical assumptions, units and boundary conditions should be exposed rather than hidden inside implementation details.

4. Geological and Excavation Model

The geology engine represents the tunnel face as a spatially varying environment. It should support heterogeneous and mixed-face conditions and allow geological state to evolve as the TBM advances.

- Uniaxial compressive strength (UCS)
- Rock mass rating (RMR)
- Rock quality designation (RQD)
- Abrasivity
- Fracture density
- Rock density
- Groundwater and water pressure
- Rock temperature
- Face heterogeneity and geological transitions

5. TBM Configuration and Engineering

The machine is configuration-driven. The platform should support EPB, slurry, hard-rock, single-shield, double-shield and mixed-face TBM architectures.

```
tbm:
  type: hard_rock
  diameter_m: 7.2
  cutterhead_rpm_max: 8.0
  thrust_max_kn: 45000
  torque_max_knm: 12000
  installed_power_mw: 6.5
```

Given tunnel diameter, tunnel length, geology, production targets and site constraints, the engineering engine should produce candidate configurations and calculate expected machine capability, power, thrust, torque, penetration, energy intensity, wear and maintenance requirements.

6. Manufacturer and Factory Simulation

TBM Foundry models the manufacturer itself. This distinguishes it from a pure tunnelling simulator. The virtual machine should exist simultaneously as an engineering configuration, a bill of materials, a factory assembly object, a tested machine and a field asset.

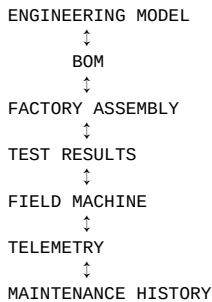
```
CUSTOMER REQUIREMENTS
↓
TBM CONFIGURATION
↓
ENGINEERING MODEL
↓
PERFORMANCE SIMULATION
↓
MANUFACTURING CONFIGURATION
↓
FACTORY BUILD
↓
FACTORY ACCEPTANCE TEST
↓
DEPLOYMENT
```

- Raw material preparation
- Machining
- Welding
- Assembly
- Electrical integration

- Hydraulics
- System integration
- Factory acceptance testing
- Quality assurance
- Shipping and commissioning

The factory simulator should answer questions concerning production lead time, component bottlenecks, inventory requirements, workstation utilisation, component lead times and testing schedules.

7. Manufacturing Digital Twin



A persistent machine identifier should bind these representations together, creating a traceable lifecycle record from engineering conception through field use.

8. Kotlin Digital-Twin Dashboard

Kotlin is the primary visual and interactive layer. It should not duplicate the physical simulation; instead it should render authoritative state supplied by the Python/Rust backend.

- TBM 2D/3D visualisation
- Cutterhead and cutter animation
- Shield and articulation state
- Tunnel and geology rendering
- Muck-flow animation
- Hydraulic visualisation
- Factory animation
- Live telemetry
- Engineering charts
- Production and maintenance dashboards

Kotlin project structure

```

kotlin_dashboard/
├── build.gradle.kts
├── settings.gradle.kts
├── src/main/kotlin/com/tbmfoundry/
│   ├── Main.kt
│   ├── app/
│   ├── dashboard/
│   ├── tbm/
│   ├── factory/
│   ├── telemetry/
│   ├── geology/
│   ├── navigation/
│   ├── network/
│   └── models/
  
```

■■■ components/

The visual layer should behave as a digital twin: changing thrust should alter cutterhead loading, torque, vibration, material flow, temperature and telemetry in a visually coherent manner.

9. R Engineering Analytics

R is an analytical subsystem rather than a control system. It provides statistical interpretation of simulation and field data.

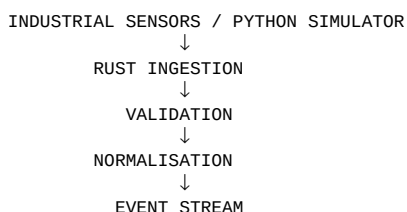
- Production statistics
- Penetration analysis
- Cutter-life modelling
- Energy-intensity analysis
- Failure distributions
- Regression and geological correlation
- Forecasting
- Reliability analysis
- Confidence intervals

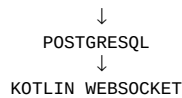
```
r_analytics/  
■■■ DESCRIPTION  
■■■ NAMESPACE  
■■■ R/  
■ ■■■ productivity.R  
■ ■■■ penetration.R  
■ ■■■ cutter_wear.R  
■ ■■■ energy_intensity.R  
■ ■■■ geostatistics.R  
■ ■■■ regression.R  
■ ■■■ forecasting.R  
■ ■■■ reliability.R  
■ ■■■ reports.R  
■■■ tests/
```

10. Rust Real-Time Infrastructure

Rust provides the high-throughput industrial data path: telemetry ingestion, event processing, WebSockets, validation, machine-state services and selected performance-critical numerical functions.

```
rust_backend/  
■■■ Cargo.toml  
■■■ crates/  
■ ■■■ tbm_core/  
■ ■■■ telemetry/  
■ ■■■ events/  
■ ■■■ machine_state/  
■ ■■■ factory_state/  
■ ■■■ spatial/  
■ ■■■ database/  
■ ■■■ api/  
■ ■■■ analytics_bridge/  
■■■ services/  
■ ■■■ telemetry_service/  
■ ■■■ digital_twin_service/  
■ ■■■ event_service/  
■ ■■■ api_service/  
■■■ tests/
```





11. PostgreSQL / PostGIS Data Model

PostgreSQL is the persistent system of record; PostGIS provides spatial and geological capabilities.

- Machines
- TBM configurations
- Components and cutters
- Geology and tunnels
- Factory lines and workstations
- Bills of materials
- Inventory
- Production orders
- Telemetry and events
- Maintenance
- Factory tests
- Missions
- Projects
- Performance records
- Analytics results

Immutable event model

TBM_CREATED
 TBM_CONFIGURED
 COMPONENT_STARTED
 MACHINING_STARTED
 ASSEMBLY_STARTED
 FACTORY_TEST_STARTED
 FACTORY_TEST_COMPLETED
 TBM_SHIPPED
 TBM_COMMISSIONED
 TUNNELLING_STARTED
 GEOLOGY_CHANGED
 CUTTER_REPLACED
 MAINTENANCE_STARTED
 MAINTENANCE_COMPLETED
 TUNNELLING_COMPLETED

Each event should include an event ID, machine ID, timestamp, event type, payload and schema version.

12. Engineering Optimisation

Optimisation should operate within explicit machine, geological, thermal, wear, muck-handling and safety constraints.

- Cutterhead RPM
- Thrust
- Penetration
- Machine power
- Muck handling

- Cooling
- Energy per metre
- Cutter utilisation
- Production rate
- Factory assembly sequence
- Machining schedules
- Inventory
- Supplier lead times
- Testing schedules

13. Field Performance and Replay

Planned, simulated and observed machine behaviour should be directly comparable. The platform should calculate planned-versus-actual advance, energy, wear and downtime. Historical machine states should be replayable at 1×, 10× or 100× real time.

14. Simulation Modes

- Engineering Mode
- Factory Mode
- Commissioning Mode
- Tunnelling Mode
- Maintenance Mode
- Failure Mode
- Training Mode
- What-if Mode

15. Observability and Testing

- Telemetry throughput
- Simulation speed
- WebSocket latency
- Database latency
- R analytics latency
- Python simulation runtime
- Kotlin rendering performance

Testing must cover Python physics and numerical convergence; Rust telemetry, concurrency and APIs; R statistical validity; and Kotlin UI/state synchronisation and performance.

16. Development Roadmap

Phase 1. Python TBM physics engine

Phase 2. Geology and excavation

Phase 3. Factory and manufacturing simulator

Phase 4. PostgreSQL/PostGIS persistence

Phase 5. Rust telemetry and event backend

Phase 6. R analytics and calculation layer

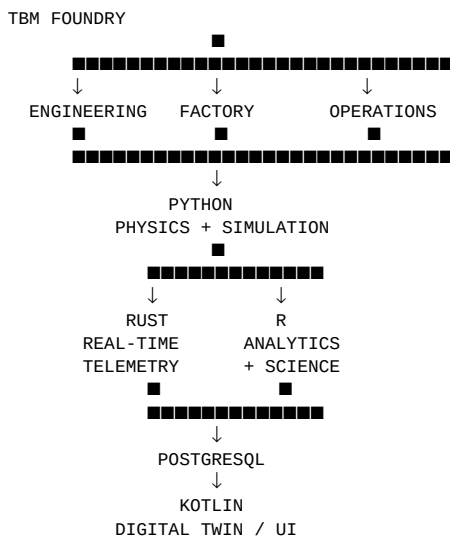
Phase 7. Kotlin digital-twin dashboard

Phase 8. Real-time synchronisation

Phase 9. Engineering optimisation

Phase 10. Scenario, Monte Carlo and advanced engineering research

17. Final Development Model



Architectural principle: Python computes what the machine is; Rust moves what the machine is doing; R explains what the machine has done and what it is likely to do; Kotlin shows the engineer what is happening.

The intended endpoint is a unified manufacturer-grade digital engineering environment connecting customer specification, TBM engineering, virtual prototyping, factory manufacture, testing, field commissioning, live tunnelling, performance analysis and subsequent machine development.

Engineering-support software only: numerical outputs should be treated as simulation results and should not be represented as certified engineering designs without independent professional validation.